

AirDesk: Mid-Air Gestures for Situationally Impaired Desktop Interaction

CADEN CALDERON, Colorado State University, USA

AirDesk is a webcam-based prototype for controlling a Hyprland Linux desktop with mid-air hand gestures. The project is motivated by situationally impaired interaction: moments when keyboard and mouse input is unavailable, undesirable, dirty, painful, or physically costly. The system does not try to replace traditional desktop input. Instead, it explores whether a small gesture vocabulary can work as a secondary control channel for coarse desktop actions and occasional mouse-like interaction. This paper describes the interaction design, MediaPipe landmark pipeline, gesture-to-action mapping, and a small two-person pilot evaluation. Early recognition experiments showed why continuous dynamic gestures are difficult in a live desktop stream, so the final prototype uses deterministic landmark logic for a smaller but more reliable mid-air mouse and window-manager grammar. In a normal desktop condition, keyboard and mouse were faster. In a dirty-hands condition where keyboard and mouse required hand washing before input, AirDesk became competitive and slightly faster on average. The paper contributes a focused system design and an evidence-bounded evaluation of mid-air desktop control under situational impairment.

CCS Concepts: • **Human-centered computing** → **Interaction techniques**; *User interface design*; *HCI design and evaluation methods*.

Additional Key Words and Phrases: mid-air gestures, situational impairment, desktop interaction, hand tracking, Hyprland, 3D user interfaces, multimodal interaction

1 Introduction

Keyboard and mouse input are still excellent for most desktop work. They are precise, familiar, and fast. The problem is that people do not always use desktops under ideal conditions. A user may be cooking while following instructions on a monitor, painting or soldering with dirty hands, wearing gloves, standing away from a desk during a presentation, or avoiding repeated wrist movement during a long work session. In those moments, even simple operations such as switching workspaces, pausing media, moving window focus, or cancelling a command can become awkward. A recipe video is a simple example: if the user’s hands are covered in flour or oil, rewinding ten seconds, scrolling back to an ingredient, or copying a note may require stopping, washing, touching the computer, and then returning to the food.

AirDesk explores whether webcam-based mid-air hand gestures can help in these situations. The goal is not to replace keyboard and mouse input during normal desktop work. Instead, AirDesk focuses on a secondary control loop where availability matters more than maximum speed: moving between workspaces, moving a pointer, clicking, scrolling, selecting, copying, pasting, and controlling small profile-specific actions. A useful AirDesk command does not need to beat a keyboard shortcut when someone is already sitting at the keyboard. It needs to be available when touching the keyboard or mouse is inconvenient.

This paper presents AirDesk as a system paper with pilot evidence. The prototype is a Python-based webcam-to-desktop pipeline for Hyprland Linux. It includes camera capture, MediaPipe hand tracking, normalized hand-state events, deterministic control policy, practice/logging mode, guarded desktop actions, pointer event injection, replayable logs, labeling tools, feature export, and offline recognizer evaluation. The evaluation is intentionally formative: I tested the system with one informal participant to identify which parts of the interaction are promising and which parts are not yet reliable enough for live desktop control.

This paper makes three contributions. First, it describes an interaction design for mid-air desktop control based on a small vocabulary, explicit intent, and visible recovery behavior. Second, it documents a prototype pipeline that records,

Author’s Contact Information: Caden Calderon, Colorado State University, Fort Collins, Colorado, USA, cadence@colostate.edu.

Manuscript submitted to ACM

replays, labels, and evaluates hand-tracking sessions without repeatedly re-performing gestures. Third, it reports pilot evidence showing when a practical mid-air mouse and window-manager workflow is useful, especially when ordinary input has a cleanup or access cost.

2 Related Work

2.1 Situational and Ability-Based Input

Situationally induced impairments and disabilities describe contexts where environmental or temporary constraints make ordinary input harder to use [9]. Much SIID work is mobile-centered, but the idea also fits desktop computing. A keyboard and mouse may be physically present, yet still be a poor fit when a user’s hands are dirty, gloved, occupied, painful, or far away from the desk. AirDesk uses this framing to motivate optional desktop gestures without treating them as universally better input.

Ability-based design provides the accessibility frame for this project. Rather than assuming one ideal input method, ability-based systems adapt to what users can do in a given context [14]. AirDesk is best understood as an additional ability-aware input mode: it gives the user another way to act when the normal way is temporarily a poor fit. The pilot focuses on that situational idea rather than making broad claims about disability access.

2.2 Mid-Air Gesture Design

Prior work on mid-air interaction shows both the appeal and difficulty of gesture input. Mid-air gestures can be expressive and easy to demonstrate, but their usability depends heavily on the application, task, evaluation method, feedback, training, prior experience, and participant factors [3, 7]. Gesture elicitation work also pushes against the idea of a universal mid-air vocabulary. Vogiatzidakis and Koutsabasis report that gesture identification depends on context of use [10], while Aigner et al. show that preferred gesture types vary by intended meaning and action [1]. Work on wall-display gestures makes the same point in another setting: the display and task shape the gesture set [13].

AirDesk applies these findings by choosing a small vocabulary for desktop control instead of importing a broad gesture set from another domain. Workspace navigation maps naturally to directional movement, clicking and dragging map to pinch, and higher-risk window actions need deliberate clutching or combos. The design goal is not to find a universal vocabulary. It is to choose a vocabulary that is plausible, learnable, and measurable for AirDesk’s desktop setting.

2.3 Interaction Costs and Intent

Mid-air input also has real costs. Hincapie-Ramos et al. show that upper-limb fatigue is a measurable design issue in mid-air interaction [4]. Error-prone in-air recognizers can affect user behavior and perceived control, especially when errors come from both human performance and system ambiguity [2]. These concerns matter for desktop control because a false activation can change the user’s actual workspace.

Prior systems have handled unintended activation by separating activation gestures from command gestures. For example, mid-air smart-home control work uses registration gestures to decide which device should respond before accepting commands [11]. Public-display selection work also emphasizes clear phases and immediate feedback [12]. AirDesk uses the same basic idea through stable pose events, command clutches, and cooldowns. Continuous hand motion should not automatically become continuous high-impact command input.

Work comparing touch and mid-air interaction also supports AirDesk’s secondary-channel framing. Jakobsen et al. found that touch can outperform mid-air for direct target acquisition, but users become more willing to choose mid-air as the movement cost of touch increases [6]. AirDesk follows that logic for desktop input. The important question is not whether a gesture beats a keyboard shortcut under ideal desk conditions. The question is whether a gesture becomes useful when ordinary input becomes costly.

2.4 Recognition and Implementation Background

AirDesk uses MediaPipe Hand Landmarker as its first hand-tracking backend [16]. MediaPipe provides practical RGB hand landmarks, but it is not the contribution of this paper. The contribution is the desktop-control system built around tracking: safety policy, mode handling, replayable logs, deterministic landmark logic, and evaluation.

For gesture recognition, AirDesk explored static pose primitives, rule-based dynamic phrases, template or DTW matching, deterministic motion spotting, and learned temporal models. Template-based recognizers have a long history in HCI prototyping, including Wobbrock et al.’s work comparing simple recognizers with DTW and other baselines [15]. Temporal convolutional networks are useful for action segmentation and detection because they can model evidence over time rather than treating each frame in isolation [8]. AirDesk uses this distinction to keep model claims honest: learned models remain a promising future direction, but the current live-control prototype uses direct landmark logic because it was more reliable under the course timeline.

A key recognition challenge is the sliding-window problem. Many prototype recognizers classify a fixed window of recent frames. That works best when the gesture has already been neatly clipped, but live desktop control is not neatly clipped. A window may contain only the first half of a swipe, the end of one gesture plus the beginning of the next, or ordinary hand motion that happens to look gesture-like. Increasing the window size can capture more context, but also adds latency and makes repeated gestures blur together. Decreasing it improves responsiveness, but makes partial gestures more likely. AirDesk’s TCN v2 direction is motivated by this trade-off: the model can still use a causal rolling window as context, but the window should not be treated as the gesture itself.

3 Design Goals and Research Questions

AirDesk is built around four design goals.

G1: Support practical desktop control without pretending to replace the mouse. The current target actions are pointer movement, click, drag, scroll, workspace movement, moving windows between workspaces, and a few profile-specific commands. These actions expose most of the system without requiring broad text entry or fine-grained drawing precision.

G2: Require explicit user intent before high-impact actions. The system should not interpret every visible hand movement as a command. AirDesk uses stable pose events, fist clutching, cooldowns, and deliberate combos so that workspace movement or window actions are harder to trigger by accident.

G3: Make mistakes easy to see and recover from. Gesture errors are more serious when they control a real desktop. AirDesk therefore supports a practice/logging mode, records actions, and keeps higher-risk commands behind clearer gestures.

G4: Make recognition behavior replayable and measurable. Gesture systems are easy to demo and hard to trust. AirDesk records hand-state streams, runtime events, labels, features, and evaluation outputs so recognizers can be tested against the same sessions. This is especially important for continuous gesture spotting, where window accuracy can look good even when event timing, repeated gestures, or background rejection fail.

These goals lead to the following research questions for the formative pilot:

- (1) Can a small deterministic hand-control vocabulary support a realistic desktop workflow with acceptable false activation, latency, and perceived control?
- (2) Which parts of the workflow are promising for mid-air input, and which remain slower or more awkward than keyboard and mouse?
- (3) How does gesture control compare with keyboard and mouse input when ordinary input is immediately available versus when it is temporarily costly to use?

4 System Overview

AirDesk is implemented as a Python prototype with a modular pipeline. Hyprland is the initial desktop target because it exposes workspace and focus commands through dispatcher actions [5].

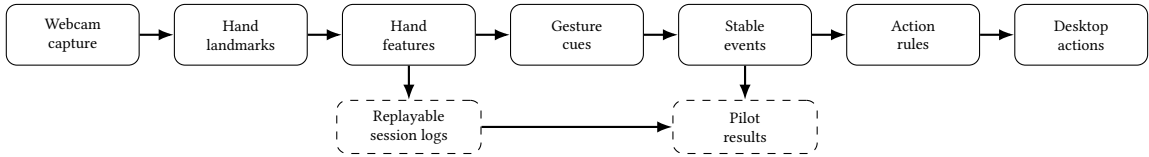


Fig. 1. AirDesk’s current architecture. Webcam frames become hand landmarks, gesture cues, stable events, and finally desktop actions. The same streams can be saved for replay and pilot analysis.

The current prototype includes OpenCV camera capture, MediaPipe hand tracking, replayable session logs, deterministic control poses, stable pose debouncing, a short combo buffer, practice/logging mode, guarded Hyprland commands, Linux pointer events, label files, feature export, and offline gesture evaluation. Figure 1 shows the main architecture.

4.1 Pipeline Components

The capture layer reads webcam frames through OpenCV. The first tracking backend uses MediaPipe to infer hand landmarks from RGB frames. AirDesk converts those landmarks into normalized hand-state records with timing, hand presence, landmark positions, and derived pose features. This normalization step keeps the rest of the system from depending too tightly on MediaPipe. Future depth cameras or alternative hand trackers could emit the same hand-state shape.

The control layer converts hand states into observable facts: open or relaxed hand, fist, index pinch, middle pinch, palm zone, vertical movement, and ambiguity signals. A debouncer turns noisy frame-level pose facts into stable enter, hold, and release events. A small grammar then maps those events to pointer movement, clicks, scroll, workspace movement, window movement, launcher activation, or close-window combos. Learned and DTW recognizers remain available in the project as replay and diagnostic tools, but they are not the live action path for the final prototype.

4.2 Landmarks to Control Events

The live prototype uses MediaPipe’s hand landmarker output as geometry rather than as a gesture classifier. For each visible hand, MediaPipe reports a standard 21-point hand skeleton with normalized coordinates for the wrist, thumb joints, finger joints, and fingertips. AirDesk stores these points in a tracker-independent hand record with a palm center, bounding box, handedness, and confidence score. The important design choice is that later code does not ask MediaPipe to decide what a command means. AirDesk computes its own pose facts from the landmark geometry.

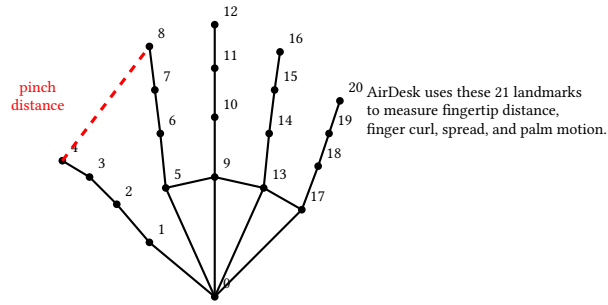


Fig. 2. Simplified 21-landmark hand model used by MediaPipe. In the prototype, a pinch is detected by measuring normalized distance between the thumb tip and a fingertip, while fist/open-palm evidence uses finger curl and spread.

Pinch detection is the simplest example. The system measures the distance from the thumb tip to the index fingertip for index pinch, and from the thumb tip to the middle fingertip for middle pinch. If that normalized distance falls below a threshold, the pose becomes a pinch candidate. A short pinch release becomes a click; a pinch held long enough or moved far enough becomes a drag; a middle pinch with vertical palm motion becomes scroll. For example, selecting and copying text in the pilot meant moving the open hand to the text, using index pinch to click or drag over it, using middle pinch to scroll when needed, then using a right-click gesture to open the copy menu.

Fist and open-palm detection use more evidence because they are higher-impact controls. Open palm combines extended-finger count and finger spread. Fist combines finger curl, fingertip clustering, thumb support, low open-palm score, and palm-relative scale. This is more robust than a single threshold such as “finger tips below knuckles,” because webcam angle and hand distance can distort one measurement. Even with multiple signals, the prototype treats ambiguous frames conservatively. If a hand looks partly like a fist and partly like a pinch, or if index and middle pinch are both plausible, the control layer can suppress the frame rather than firing both commands.

The raw pose facts are still too noisy to become commands directly, so the system waits for a pose to be stable across frames. In practice, this means one noisy frame does not immediately click or switch workspaces. The system emits simple events such as entered, held, and released. A quick pinch enter-and-release becomes a click. A pinch that stays active becomes a drag. A fist that stays active creates an anchor point, and only then does motion away from that anchor become a workspace or window command. This design gives the user a clutch: the fist means “the next motion is a window-manager command,” while ordinary open-hand motion remains pointer control.

5 Interaction Design

5.1 Control Mode and Safety

Safety is central because false activations can affect the real desktop. The prototype can run in a practice mode that records what it would do before sending real commands. The current control runtime treats ordinary hand movement as pointer movement, while higher-impact actions require clearer poses or held movement. Fist is the main command clutch for workspace and window-manager actions. Destructive commands, such as closing a window, are mapped to deliberate combos rather than a single accidental pose.

The clutch is also an interaction design choice. It gives the user a visible boundary between ordinary motion and command motion. This matters in a webcam setting, where normal desk motion such as reaching for a phone, adjusting posture, or moving near the keyboard can resemble parts of a command gesture.

5.2 Gesture Vocabulary

The current command vocabulary is intentionally small:

- relaxed or open hand: move the pointer;
- index pinch tap: left click;
- index pinch hold and move: drag or select;
- middle pinch hold and vertical movement: scroll;
- middle pinch tap: right click;
- fist held and moved up/down: switch workspaces;
- fist held and moved left/right: move the active or targeted window to another workspace;
- open palm, fist, open palm: deliberate close-window combo;
- open palm followed by sideways palm: launcher command.

This vocabulary emphasizes a small desktop workflow rather than a large symbolic gesture language. The mappings were chosen to make the gesture meaning visible and to separate low-risk continuous control from higher-risk window-manager commands. Open or relaxed hand movement controls the pointer because it is the least effortful resting pose and can be used continuously. Pinch maps to click because bringing two fingers together has a clear commit feeling and is already familiar from touch, mobile, and spatial interfaces. Holding that same pinch naturally extends into drag or selection. Middle pinch keeps scrolling and right click separate from normal left click while staying in the same pinch family.

Fist is used differently. It acts as a clutch for workspace and window movement because it is visually distinct from an open hand and feels closer to grabbing or carrying. That makes it a better fit for commands that change the desktop state. Closing a window is even riskier, so AirDesk maps it to an intentionally deliberate open-palm, fist, open-palm combo rather than to a single pose. These choices reflect the interaction goal: common actions should be quick, but destructive or disorienting actions should require clearer intent.

The system is also multimodal in a practical desktop sense. It does not ask gestures to do every job. It combines camera-based hand input, pointer emulation, Hyprland window-manager commands, visible runtime feedback, practice logs, and ordinary keyboard and mouse fallback. This makes gestures one layer in a broader interaction loop rather than a replacement for all other input.

5.3 Feedback and Recovery

Recovery is part of the interaction, not an afterthought. A command system that can switch workspaces, move windows, or click inside documents must give users a way to stop, wait, and recover from accidental recognition. The current design includes pause behavior, cooldowns after accepted commands, ambiguity suppression, and practice logs. The pilot pays attention not only to whether intended commands fire, but also to whether participants feel they can keep the system under control.

6 Implementation and Reproducibility

6.1 Runtime and Profiles

The implementation is organized as a Python command-line prototype. Runtime commands can capture live webcam input, replay recorded sessions, route recognized events to practice logs, or invoke guarded Hyprland dispatchers.

Profiles define which gestures are active, how they map to actions, and whether the system is operating in practice mode or execution mode.

The default testing posture is practice-first. This lets the prototype show what would have happened without actually changing the desktop. For live execution, the dispatcher layer is intentionally narrow: only explicit actions such as workspace movement, window movement, focus movement, pointer movement, click, drag, and scroll are connected during the pilot.

6.2 Logging, Replay, and Labeling

AirDesk records normalized tracking frames and runtime events as JSONL. Replay support allows recognizer changes to be evaluated without repeatedly performing gestures. The labeling workflow creates per-recording JSON label files with event and phase labels, then exports deterministic landmark-derived feature CSVs for evaluation and future model training.

This replay workflow makes the system easier to evaluate honestly. A recognizer can be changed, tuned, or replaced, then tested against the same recorded sessions. For a prototype gesture system, that matters because a short live demo can look successful even when the same recognizer fails on repeated commands, background motion, or slightly different hand positions.

6.3 Recognizer Evolution

AirDesk’s recognizer changed as the project moved from isolated gestures toward continuous desktop use. This shift was partly motivated by prior project experience with an LSTM-based smart-home gesture recognizer. That recognizer used a traditional sliding window over recent frames. It could work when the window happened to contain a full gesture, but it was fragile when the timing was not perfect: the window might capture only part of a gesture, include the reset motion afterward, or contain the end of one gesture and the start of the next. It was also slower than desired for interactive control.

That experience shaped AirDesk’s model search. The goal was not just a higher-accuracy classifier, but a recognizer that could handle fast commands, repeated gestures, and short chains without requiring a long pause between actions. DTW and TCNs became the two practical directions. DTW can align variable-length motion traces with a small number of personal examples, but it still needs the system to decide where a gesture starts and ends. In continuous use, that boundary problem is often the hard part.

TCN v2 was a response to the same boundary problem. The first learned AirDesk model still treated rolling windows too much like gesture clips: each window received one label, and the model tried to classify the window. That is a weak match for live desktop gestures, where the stream contains partial gestures, pauses, resets, fast repeated commands, tracking loss, and ordinary background motion. TCN v2 instead treats a rolling window as compute context. The model can produce frame-level evidence such as intentional motion, stroke direction, and start/end boundaries, while a separate decoder turns that evidence into one command event.

In practice, learned recognition still did not become reliable quickly enough for the class prototype. Rule-based swipes were inspectable but brittle under natural motion. DTW was promising as a personalized research baseline, but still depended on segmentation. IPN Hand and TCN models produced useful diagnostic evidence, but live preview still showed unsafe activations from ordinary open-hand presence, posture changes, and non-command movement. That result changed the implementation goal. Instead of forcing a learned recognizer into live Hyprland control, AirDesk

moved the learned stack into a replay and future-work lane and used deterministic landmark logic for the working prototype.

6.4 Deterministic Control Pivot

The final prototype uses deterministic landmark logic for live control. This was not the original plan, but it was the more practical design after the learned-recognition experiments. The system now reads direct MediaPipe facts such as fingertip distances, finger curl, fingertip clustering, palm position, palm zone, and vertical movement. Those facts are converted into stable poses such as relaxed hand, fist, index pinch, and middle pinch. The grammar then maps those stable events into desktop actions.

This pivot also made the interaction more legible. A user does not need to learn whether a hidden model classified a window correctly. They can see that an open hand moves the cursor, a pinch clicks, a middle pinch scrolls, and a fist becomes a window-manager clutch. The system still uses thresholds, smoothing, cooldowns, and ambiguity suppression, but the behavior is easier to tune and explain than a multi-class learned model under time pressure.

7 Pilot Method

7.1 Participants

The pilot included the author and one non-developer informal participant. This is a small formative pilot, not a generalizable user study. The participant count reflects the course timeline and solo-project constraints. The second participant is treated as an informal pilot participant rather than as a project author.

7.2 Apparatus

The pilot ran on my Linux laptop using the built-in laptop webcam and the AirDesk prototype. The camera was configured at approximately 640 by 480 pixels and 30 FPS. The session was run in good indoor lighting to give MediaPipe the best practical tracking conditions available for the prototype. Hyprland was the live desktop target. This setup is intentionally modest: it represents the kind of camera already available on many laptops, but it also makes tracking sensitive to lighting, distance, and camera angle.

7.3 Conditions and Tasks

The pilot compared the same task under two contexts. The first context was normal desktop use, where keyboard and mouse could be used immediately. The second context simulated dirty-hand interaction. Participants covered their hands with a messy mixture of olive oil, flour, and honey. In the keyboard and mouse condition, participants first washed their hands, then used the keyboard and mouse. In the AirDesk condition, participants skipped washing and completed the task using hand gestures.

The input conditions were:

- keyboard and mouse baseline at the desk;
- AirDesk hand-only control using the deterministic control runtime.

The task was designed to expose most of the current gesture vocabulary in one realistic workflow. The participant started on workspace 1. A Google Docs tab was open on workspace 8. The goal was to navigate to that workspace, click the correct document, scroll to page 4, select a keyword/code string, highlight or copy it, return to workspace 4, click into a second Google Docs tab, paste the code into a “paste here” location, and stop the run. This task requires

workspace navigation, cursor movement, clicking, scrolling, text selection, right click or copy behavior, returning across workspaces, and paste placement. Figure 3 summarizes the flow used for every timed run.

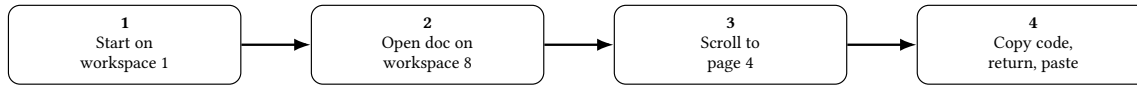


Fig. 3. Pilot task workflow. The same sequence was used in the normal desktop and dirty-hands conditions so that the comparison focused on input context rather than task differences.

The workflow was chosen because it is closer to an interruption task than to a gesture checklist. The participant has to move between contexts, recover a small piece of information, manipulate a document, and return to another workspace. That pattern matches the project motivation: a user may not want to perform a long desktop session with gestures, but may need to quickly retrieve, scroll, copy, paste, or control something when normal input is temporarily costly.

7.4 Procedure

The second participant first explored the gestures and workflow for about 5–10 minutes until the controls felt understandable. In the normal desktop comparison, I first demonstrated the workflow, then both participants completed two keyboard and mouse runs followed by two AirDesk hand-control runs. The dirty-hands comparison repeated the same workflow with two keyboard and mouse runs and two AirDesk hand-control runs per participant. Participants were told that the system was a prototype and could be stopped at any time. The session would stop if false activations repeatedly occurred during normal motion, if the participant reported fatigue or discomfort, or if live execution controlled the wrong target more than once.

7.5 Measures

Objective measures include task success, completion time, false activations, missed intended gestures, repeated fires, fallback keyboard or mouse use, and tracking loss. Subjective measures include perceived control, fatigue or discomfort, frustration, perceived usefulness, preference versus keyboard and mouse baseline, and short interview notes.

8 Pilot Results and Findings

The results focus on the working interaction technique: deterministic landmark control for a realistic desktop task. Earlier model experiments explain why the final prototype used direct landmark logic, but the pilot itself evaluates the system that participants actually used.

8.1 Two-Person Pilot

In the normal two-person pilot workflow, keyboard and mouse were faster. Across both participants, the keyboard and mouse condition averaged 22.0 seconds, while the hand-control condition averaged 41.5 seconds. This result was expected. AirDesk is not designed to beat a mouse and keyboard when the user is already seated at the desk with both devices available. The more important result is that the hand-control condition completed the same multi-step workflow using only mid-air gestures.

The pilot also clarified where AirDesk is most useful. For normal desktop use, mouse and keyboard remain faster and less tiring. For situationally impaired use, the hand-control workflow becomes more meaningful because the alternative

Table 1. Normal desktop pilot comparison for the Google Docs workspace task. Keyboard and mouse were available immediately.

Participant	Keyboard/mouse runs	AirDesk runs	Mean difference
Author	22s, 20s	41s, 39s	AirDesk 19.0s slower
Participant B	24s, 22s	44s, 42s	AirDesk 20.0s slower
Overall	22.0s mean	41.5s mean	AirDesk 19.5s slower

may not be a perfect keyboard shortcut; it may be touching the computer with dirty hands, leaving a task to return to the desk, or being unable to comfortably use the mouse. In that setting, a slower hands-free command can still be useful.

8.2 Dirty-Hands Condition

The dirty-hands condition better matched AirDesk’s motivating use case. Participants began with messy hands using olive oil, flour, and honey. For keyboard and mouse runs, they had to wash their hands before using the devices. For AirDesk runs, they could keep their hands dirty and complete the task with gestures. This made the comparison less about whether mid-air input can beat a mouse in ideal conditions and more about whether it can avoid the transition cost of cleaning up before touching the computer.

Table 2. Dirty-hands pilot results. Keyboard/mouse time includes washing hands before using the devices. AirDesk time skips washing and uses hand control.

Participant	Keyboard/mouse runs	AirDesk runs	Mean difference
Author	46s, 43s	38s, 40s	AirDesk 5.5s faster
Participant B	44s, 42s	40s, 42s	AirDesk 2.0s faster
Overall	43.8s mean	40.0s mean	AirDesk 3.8s faster

The margin changed substantially between contexts. In normal use, hand control was about twice as slow as keyboard and mouse. With dirty hands, the keyboard and mouse condition added cleanup time, and the difference nearly disappeared. AirDesk was slightly faster overall because it let participants act immediately instead of washing, returning to the computer, completing the task, and then getting their hands dirty again. This supports the central project claim: mid-air gestures are not generally better than keyboard and mouse, but they can be useful when ordinary input has a real access cost.

The practical comparison is therefore not just “gesture versus mouse.” It is “gesture now” versus “stop the surrounding task, clean up, interact with the computer, and resume.” AirDesk becomes more compelling when that transition cost is part of the real workflow.

8.3 Qualitative Observations

Both participants completed the workflow in both contexts, but the interaction did not feel equally strong across all gestures. Cursor movement and pinch-to-click felt the most natural. They are direct, visible, and similar to ordinary pointing behavior: move the hand to move the cursor, pinch to commit. Dragging and text selection were usable but required patience because the prototype still depends on stable tracking and clean pinch timing.

Scrolling was the hardest part of the workflow. The middle-pinch scroll gesture worked, but it was less familiar than clicking and more sensitive to vertical hand motion. Participants sometimes had to adjust their hand position to get the scroll direction and amount they wanted. Pinch-to-click also occasionally missed on the first attempt, requiring a

second pinch. These misses did not stop the task, but they explain part of the time difference between AirDesk and keyboard and mouse in the normal condition.

The pilot also reinforced how much webcam-based landmark control depends on setup. Good lighting and a front-facing hand improved tracking. Darker lighting, angled hands, or a hand too far from the camera made the landmarks less stable. When the hand was far away, the landmarks compressed together in the image, which made distance thresholds such as pinch more fragile. These observations shaped the limitations and future-work sections: AirDesk can work as a prototype under favorable conditions, but robust everyday use would need better calibration, depth information, or a recognizer that adapts to distance and camera angle.

8.4 Research Question Summary

The pilot gives formative answers to the research questions. For RQ1, the small deterministic vocabulary was sufficient to complete a realistic desktop workflow, but only under favorable lighting and with practice. For RQ2, cursor movement, clicking, workspace navigation, and the dirty-hands use case were the most promising; scrolling, selection, and occasional missed pinches remained slower or more awkward. For RQ3, keyboard and mouse input was clearly faster when ordinary input was immediately available, but the comparison changed when ordinary input required cleanup first. In that situationally impaired context, AirDesk became competitive and slightly faster on average.

9 Discussion

The pilot supports the central framing of the project: mid-air gestures are most useful when they are treated as an optional interaction layer, not as a universal mouse replacement. Under normal desk conditions, keyboard and mouse were clearly faster. That result is not a failure of the design. It shows that the comparison only becomes interesting when the surrounding situation changes the cost of ordinary input.

The dirty-hands condition is the clearest example. In that context, the interaction problem is not just pointing or clicking. It is the interruption around pointing or clicking: leaving the task, washing hands, using the computer, and then returning to the task. AirDesk helped because it removed that transition. The current prototype is still slower and rougher than normal desktop input, but it can be useful when the alternative includes extra physical steps outside the computer.

The design also suggests a useful boundary for future development. Mid-air input works best when it is optional and task-specific. Cursor movement, click, drag, and scroll are possible, but they are slower and more tiring than a real mouse during normal use. The strongest future versions should therefore focus on moments where a small number of gestures can remove a larger interruption, not on replacing every precise desktop action.

Finally, the small pilot should be interpreted as a formative design check. The goal is not to prove that AirDesk is finished. The goal is to identify the smallest credible desktop gesture loop that works well enough to justify broader evaluation, and to understand which parts of the workflow still feel slower, tiring, or less precise than ordinary input.

10 Limitations

This project has several limitations. The pilot participant count is small, and one participant is the author. The current hardware baseline is a built-in laptop webcam at approximately 30 FPS, which limits tracking quality under poor lighting, occlusion, fast movement, unusual camera angles, or larger camera-to-hand distances. The pilot was intentionally run in good lighting, so the results should not be read as evidence that the system works equally well in dark rooms or cluttered backgrounds. The evaluation is formative and may overfit to one setup, one workspace layout, and one user's

gesture style. The dirty-hands condition uses a simple simulated mess rather than a full cooking or workshop task, so it captures only one kind of situational impairment. Accessibility benefits are design motivations, not validated claims.

The recognition approach also has limits. The final prototype uses deterministic landmark math, which is easier to explain and tune than a learned model, but it depends on camera geometry and threshold choices. Hand distance, hand rotation, and lighting can change the landmark layout enough to affect pinch or fist detection. Future studies should add more participants, more camera setups, natural background motion, and stricter timing logs before making stronger reliability claims.

11 Future Work

Future work includes a larger participant study, improved continuous-session labeling, a stronger continuous-spotting recognizer, richer overlay feedback, more robust cursor click and drag behavior, depth or Kinect context, and profile-specific gesture vocabularies for media, presentation, and accessibility use cases. A larger study should include participants who actually experience the target constraints rather than only simulating them. For example, future work could separately study gloves, distance-from-keyboard use, wrist-pain scenarios, and presentation contexts.

The system could also support user-specific calibration. The current threshold-based approach would benefit from a short setup step that learns a user's hand size, preferred distance from the camera, and comfortable pinch/fist shapes. Longer term, DTW or TCN-style continuous recognizers may still be useful for richer gesture shortcuts, but they need better continuous data and boundary labels before they should control a live desktop.

AirDesk should also be treated as a beta prototype rather than the ceiling of the interaction. The current pilot uses direct landmark logic and a small vocabulary because that was the most reliable path for this version. A stronger learned recognizer could support richer gesture shortcuts, chained commands, and user-specific command vocabularies that reduce the number of steps in a workflow. Since the prototype already supports mouse-like control, a future version could also combine gestures with voice input for text entry and command naming. In that version, AirDesk would not only avoid dirty-hand cleanup costs; it might also become faster than keyboard and mouse for some ideal-condition workflows because complex actions could collapse into a small number of gestures or voice-plus-gesture commands.

12 Conclusion

AirDesk explores mid-air gestures as a secondary desktop control layer for situationally impaired interaction. The current prototype provides a pluggable architecture, deterministic hand-control grammar, practice-first execution, guarded desktop actions, replayable logs, labeling, feature extraction, and pilot evidence. The system is strongest when framed as an optional input channel for moments when keyboard and mouse input is inconvenient, not as a replacement for ordinary desktop input. The main lesson is that credible desktop gesture systems need explicit intent, recoverable interaction design, and realistic workflow evaluation rather than isolated gesture demos. The pilot shows that AirDesk is slower than keyboard and mouse under normal desk conditions, but becomes competitive when ordinary input requires cleanup before use.

Acknowledgments

Thanks to the informal pilot participant who helped test the prototype workflow and provided feedback on the gesture controls.

References

- [1] Roland Aigner, Daniel Wigdor, Hrvoje Benko, Michael Haller, David Lindlbauer, Alexandra Ion, Shengdong Zhao, and Jeffrey Tzu Kwan Valino Koh. 2012. *Understanding Mid-Air Hand Gestures: A Study of Human Preferences in Usage of Gesture Types for HCI*. Technical Report MSR-TR-2012-111. Microsoft Research. <https://www.microsoft.com/en-us/research/publication/understanding-mid-air-hand-gestures-a-study-of-human-preferences-in-usage-of-gesture-types-for-hci/>
- [2] Ahmed Sabbir Arif, Wolfgang Stuerzlinger, Euclides Jose de Mendonca Filho, and Alec Gordynski. 2014. How Do Users Interact with an Error-Prone In-Air Gesture Recognizer? <https://arxiv.org/abs/2105.12453> CHI 2014 Workshop on Gesture-based Interaction Design: Communication and Cognition; arXiv:2105.12453.
- [3] Justin Catalano and Yue Luo. 2025. Usable Without Touch? Revealing the Relationship Between Usability and Mid-Air Gestures. In *Proceedings of the Human Factors and Ergonomics Society Annual Meeting*, Vol. 69. 968–974. doi:10.1177/10711813251369780
- [4] Juan David Hincapie-Ramos, Xiang Guo, Paymahn Moghadasian, and Pourang Irani. 2014. Consumed Endurance: A Metric to Quantify Arm Fatigue of Mid-Air Interactions. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '14)*. 1063–1072. doi:10.1145/2556288.2557130
- [5] Hyprland. 2026. Hyprland Dispatchers. <https://wiki.hypr.land/Configuring/Dispatchers/> Accessed 2026-05-06.
- [6] Mikkel Rønne Jakobsen, Yvonne Jansen, Sebastian Boring, and Kasper Hornbæk. 2015. Should I Stay or Should I Go? Selecting Between Touch and Mid-Air Gestures for Large-Display Interaction. In *Human-Computer Interaction – INTERACT 2015*. Springer, 455–473. doi:10.1007/978-3-319-22698-9_31
- [7] Panayiotis Koutsabasis and Panagiotis Vogiatzidakis. 2019. Empirical Research in Mid-Air Interaction: A Systematic Review. *International Journal of Human-Computer Interaction* 35, 18 (2019), 1747–1768. doi:10.1080/10447318.2019.1572352
- [8] Colin Lea, Michael D. Flynn, René Vidal, Austin Reiter, and Gregory D. Hager. 2017. Temporal Convolutional Networks for Action Segmentation and Detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 156–165. https://openaccess.thecvf.com/content_cvpr_2017/html/Lea_Temporal_Convolutional_Networks_CVPR_2017_paper.html
- [9] Zhanna Sarsenbayeva, Vassilis Kostakos, and Jorge Goncalves. 2019. Situationally-Induced Impairments and Disabilities Research. In *CHI 2019 Workshop on Addressing the Challenges of Situationally-Induced Impairments and Disabilities in Mobile Interaction*. <https://arxiv.org/abs/1904.06128>
- [10] Panagiotis Vogiatzidakis and Panayiotis Koutsabasis. 2018. Gesture Elicitation Studies for Mid-Air Interaction: A Review. *Multimodal Technologies and Interaction* 2, 4 (2018), 65. doi:10.3390/mti2040065
- [11] Panagiotis Vogiatzidakis and Panayiotis Koutsabasis. 2020. Mid-Air Gesture Control of Multiple Home Devices in Spatial Augmented Reality Prototype. *Multimodal Technologies and Interaction* 4, 3 (2020), 61. doi:10.3390/mti4030061
- [12] Robert Walter, Gilles Bailly, Nina Valkanova, and Jörg Müller. 2014. Cuenesics: Using Mid-Air Gestures to Select Items on Interactive Public Displays. In *Proceedings of the 16th International Conference on Human-Computer Interaction with Mobile Devices and Services (MobileHCI '14)*. 299–308. doi:10.1145/2628363.2628368
- [13] Markus L. Wittorf and Mikkel Rønne Jakobsen. 2016. Eliciting Mid-Air Gestures for Wall-Display Interaction. In *Proceedings of the 9th Nordic Conference on Human-Computer Interaction (NordiCHI '16)*. Article 3. doi:10.1145/2971485.2971503
- [14] Jacob O. Wobbrock, Shaun K. Kane, Krzysztof Z. Gajos, Susumu Harada, and Jon Froehlich. 2011. Ability-Based Design: Concept, Principles and Examples. *ACM Transactions on Accessible Computing* 3, 3 (2011). doi:10.1145/1952383.1952384
- [15] Jacob O. Wobbrock, Andrew D. Wilson, and Yang Li. 2007. Gestures Without Libraries, Toolkits or Training: A \$1 Recognizer for User Interface Prototypes. In *Proceedings of the 20th Annual ACM Symposium on User Interface Software and Technology (UIST '07)*. 159–168. doi:10.1145/1294211.1294238
- [16] Fan Zhang, Valentin Bazarevsky, Andrey Vakunov, Andrei Tkachenka, George Sung, Chuo-Ling Chang, and Matthias Grundmann. 2020. MediaPipe Hands: On-device Real-time Hand Tracking. *arXiv preprint arXiv:2006.10214* (2020). <https://arxiv.org/abs/2006.10214>